

Protokoll

1. Grundidee der Struktur

Ich habe das Projekt so aufgebaut, dass die Bereiche klar getrennt sind.

Das hilft dabei, die Logik übersichtlich zu halten und später einzelne Teile leichter austauschen zu können.

- **HTTP-Schicht**
Nimmt Requests entgegen, prüft Content-Type, liest JSON, schickt Response zurück.
- **Business-Logik (Manager)**
Führt alle Regeln aus (z. B. Rating muss 1–5 sein, Titel darf nicht doppelt vergeben werden).
- **Datenzugriff (Repository)**
Speichert Daten in einfachen In-Memory-Listen. Durch Interfaces kann man später einfach eine echte Datenbank einbauen.
- **Modelle & Validatoren**
Validator prüfen Regeln, bevor ein Objekt überhaupt erstellt wird.
Die Models selbst stellen sicher, dass sie nur im gültigen Zustand existieren.

Diese Struktur hilft beim Testen, Lesen und Verstehen des Codes.

2. Architekturentscheidungen

Jede Klasse übernimmt genau eine Aufgabe:

- **Handler:** kümmert sich nur darum, HTTP zu bedienen.
- **Manager:** enthält alle fachlichen Regeln.
- **Validator:** prüft Daten, z. B. Titel-Länge, gültige Genres, Rating-Wert.
- **Repository:** speichert und liest Daten. Manager statt Direktzugriff

Die Handler rufen nie direkt Repositories oder Model-Konstrukturen auf.
Stattdessen läuft alles durch die Manager:

```
ratingManager.CreateRating(...)
```

So liegt die gesamte Logik an einer Stelle.
Man vermeidet doppelten Code und Fehler.

Statt überall im Code Bedingungen zu prüfen, habe ich für Media und Rating eigene Validatoren erstellt.

Dadurch:

- wird der Code kürzer
- konsistenter
- einfacher testbar

Jedes Repository hat ein Interface:

- IMediaRepository
- IRatingRepository
- IUserRepository

Die Implementierung dahinter ist aktuell In-Memory.

Der Vorteil:

Später könnte man einfach eine Datenbank einbauen, ohne die Business-Logik umzuschreiben.

3. Wie die Teile zusammenarbeiten

Der Router erkennt, welche Route zu welchem Handler gehört. Er kann auch Parameter aus Pfaden lesen (/api/ratings/Inception).

Der Handler:

- prüft Content-Type
- validiert den Request grob (z. B. JSON vorhanden)
- ruft die passende Business-Funktion im Manager auf
- schreibt die JSON-Antwort zurück

Der Manager:

Hier passiert die eigentliche Logik.

- doppelte Medientitel verhindern
- Bewertung darf nur 1–5 sein
- Kommentare werden getrimmt
- Zeitstempel gesetzt
- Daten werden im Repository gespeichert

Das Repository:

Speichert alle Daten einfach in Listen:

```
_entries.Add(entry);
```

Später kann man das austauschen, ohne den Rest anzupassen.

4. Unit Tests

Ich habe TDD verwendet (Failing Test -> Implement -> Refactor).

Eine Liste für alle Tests und welche bisher getestet wurden sind im Ordner unter tests.md

5. Probleme & wie sie gelöst wurden

JSON wurde nicht korrekt eingelesen

Fehler: RatingCreateRequest bekam immer null-Felder.

Grund: System.Text.Json ist case-sensitiv.

Lösung: [JsonPropertyName("mediaTitle")] etc. ergänzen.

Router konnte keine Parameter aus URLs lesen

Fehler: context.Parameters existierte nicht.

Lösung: RequestContext um Dictionary erweitert und Router um Parameter-Parsing ergänzt.

Duplicate Media Titles funktionierten nicht

Ursache: Duplicate-Prüfung war nur im Manager, der Handler hat aber direkt new MediaEntry() aufgerufen.

Lösung: Handler nutzt jetzt immer den MediaManager.

Content-Type fehlte bei Tests

Dadurch gab es einen 400-Fehler.

Lösung: In Postman korrekt gesetzt.

6. Zeitaufwand (Schätzung)

Teilbereich	Zeit
Grundgerüst (Server, Router, Context)	~5 Stunden
User/Auth-System	~10 Stunden
Media-Logik + Validator + TDD	~6 Stunden
Rating-System + TDD	~2 Stunden
Postman-Tests	~1 Stunde
Fehlerbehebung & Debugging	~7 Stunden
Protokoll, UML, Dokumentation	~1 Stunde
Gesamt: ca. 32 Stunden	

7. UML Diagramm

